

Entrega de IPS.

PRACTICA DE GIT

Moisés Montes Iglesias -----
uo296102

1. Situación inicial.

Para realizar el proyecto de pruebas debemos crear un repositorio Git que llamaré practicaGit.

A su vez, creo un proyecto que llamare demo1, que tendrá un paquete demo con una clase Demo.

1.1 Disponemos de un repositorio remoto.

Start a new repository for MoisesMontesIglesias

A repository contains all of your project's files, revision history, and collaborator discussion.

Repository name *

practicaGit

✔ practicaGit is available.

☐ Public

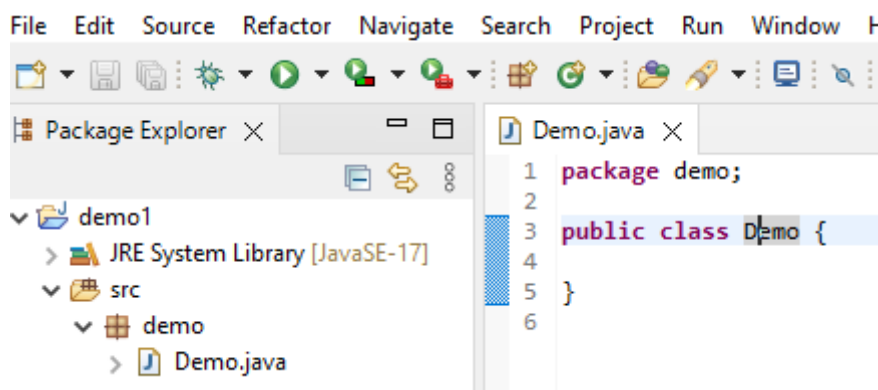
Anyone on the internet can see this repository

☒ Private

You choose who can see and commit to this repository

Create a new repository

1.3. Disponer de un proyecto para la carga inicial



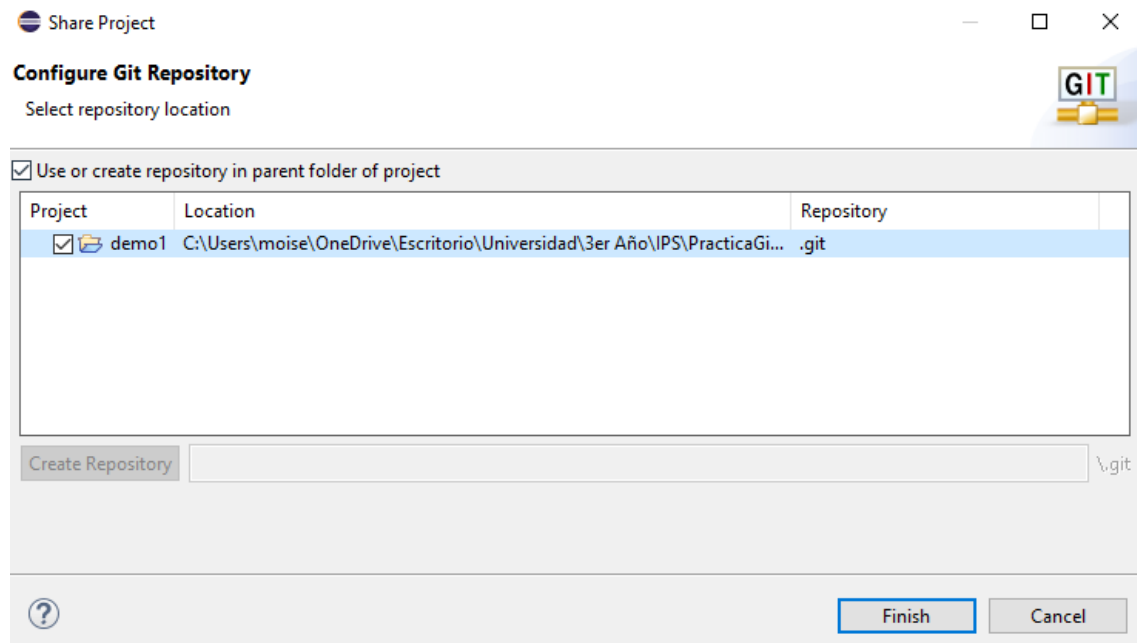
2. Conectar proyectos de Eclipse con Git.

Vamos a realizar los dos casos para conectar proyectos de Eclipse con Git.

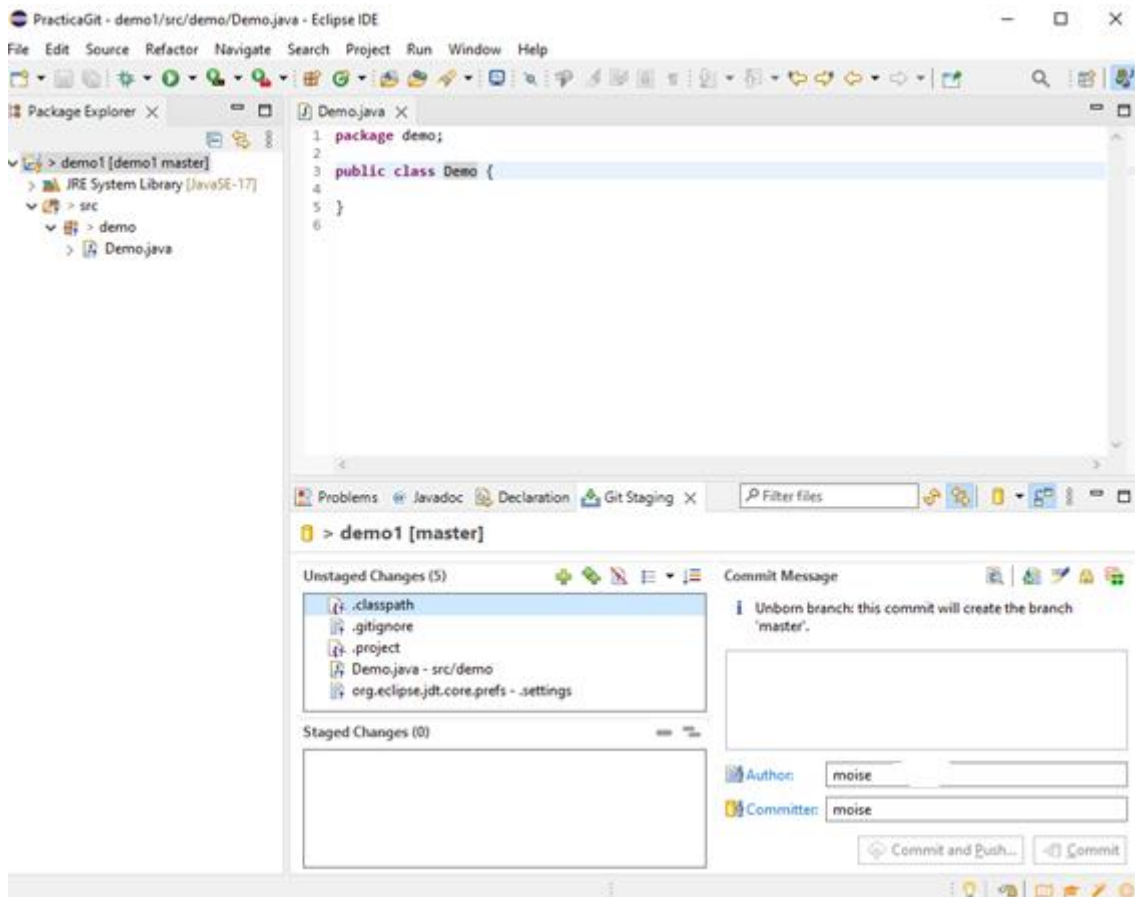
- 1) Partir de un proyecto existente, asociarlo y cargarlo en Git (Share)
- 2) Partir de un workspace en blanco, asociar y cargar un proyecto que ya existe en Git (Import)

2.1 Cargar el repositorio desde un proyecto existente.

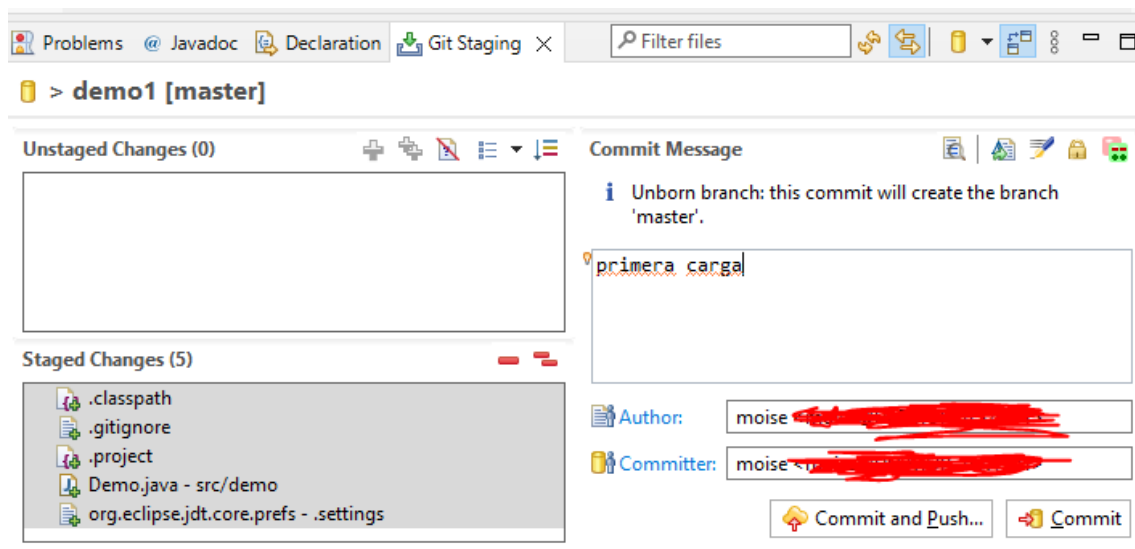
1) En eclipse seleccionamos el proyecto y creamos el repositorio local, marcando la opción que nos sale en la parte superior.



Ahora, al lado del proyecto, nos sale [demo1 master], esto indica que estamos en la rama master de nuestro proyecto.



- 2) Realizamos la primera carga en el repositorio. Vemos tanto los datos del Author y del Committer. Es obligatorio y necesario escribir un mensaje descriptivo de la carga que estamos haciendo.



- 3) Cargamos los datos también en el repositorio remoto, introduciendo la URL del repositorio que hemos creado antes

Destination Git Repository

Enter the location of the destination repository.

Remote name:

Location

URI: [Local Folder...](#)Host: Repository path:

Connection

Protocol: Port:

Authentication

User: Password: ☐ Store in Secure Store

< Back

Preview >

Push

Cancel

Push to branch in remote

Select a remote and the name the branch should have in the remote.



Source:

master f172dee primera carga

Destination:

Remote: Branch: ☒ Configure upstream for push and pullWhen pulling: ☒ Force overwrite branch in remote if it exists and has divergedShow [advanced push](#) dialog

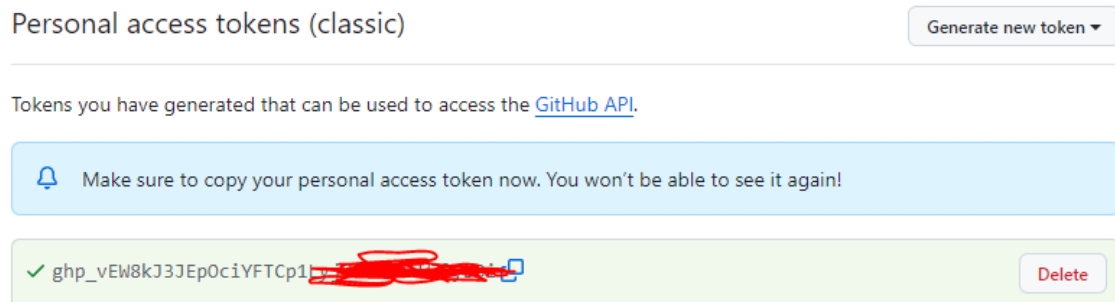
< Back

Preview >

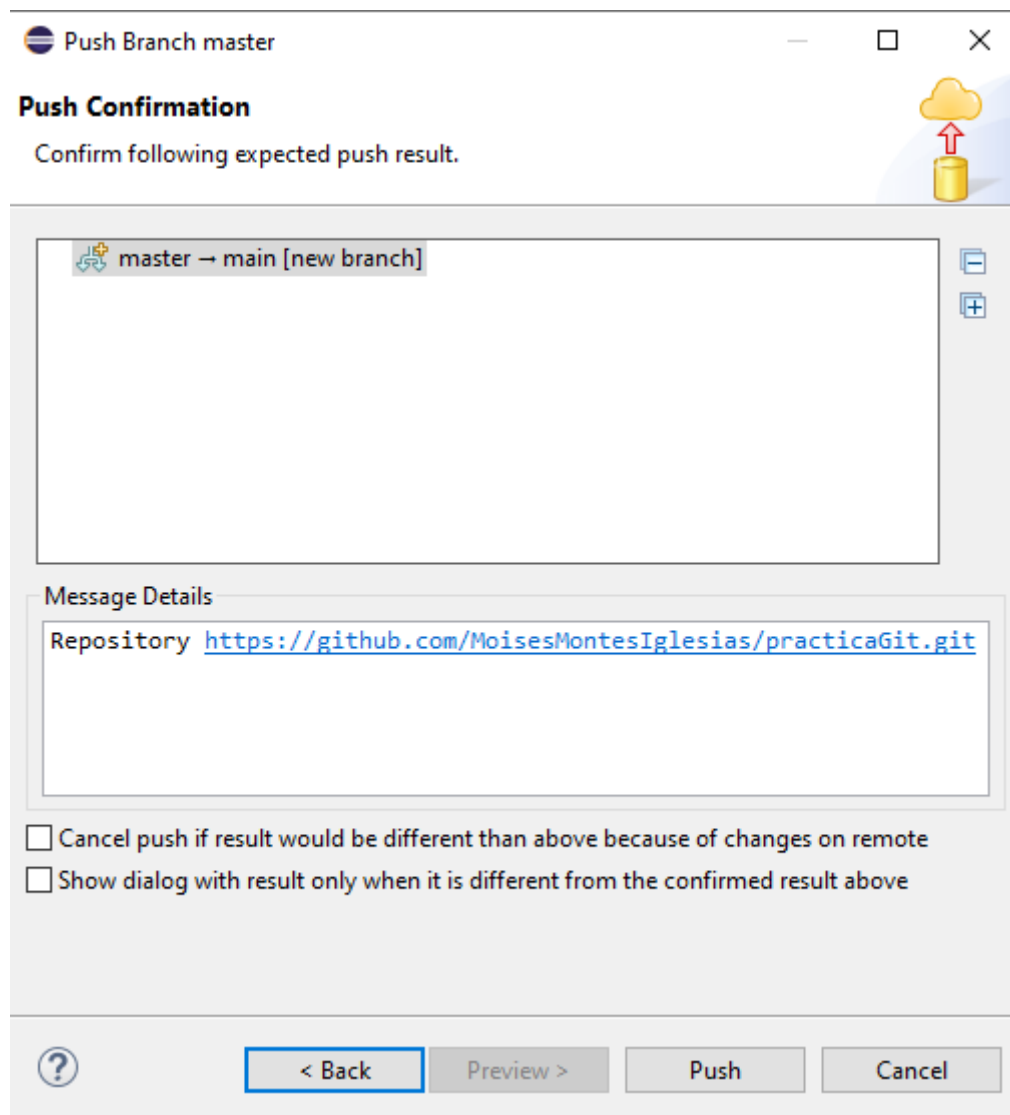
Push

Cancel

Generamos un nuevo token desde la web de Github, seleccionamos todos los opcionables de extensibles del token, ya que no sabemos cuantos podemos llegar a utilizar o necesitaremos. Este token generará un código que utilizaremos de contraseña para acceder al repositorio git.



En eclipse, confirmamos que queremos utilizar estas ramas del repositorio.



Observamos en Github que se realizó la carga que hicimos previamente.

The screenshot shows the GitHub interface for a repository named 'practicaGit', which is marked as 'Private'. At the top, there are buttons for 'Unwatch' and a notification count of '1'. Below this, a navigation bar includes a dropdown menu set to 'main', icons for repository actions, and buttons for 'Go to file', a plus sign, and a green 'Code' button. The main content area displays a commit by 'moise and moise' titled 'primera carga', made '15 minutes ago' with commit hash 'f172dee'. Below the commit header is a table of files that were added in this commit:

File	Commit Message	Time
.settings	primera carga	15 minutes ago
src/demo	primera carga	15 minutes ago
.classpath	primera carga	15 minutes ago
.gitignore	primera carga	15 minutes ago
.project	primera carga	15 minutes ago

Below the file list, a 'README' file is also visible.

2.2. Cargar un proyecto existente en el repositorio hacia eclipse.

Buscamos cargar un proyecto desde Git a nuestro entorno para poder trabajar con ello.

Para eso rellenamos con la URL del directorio donde queremos importar el proyecto.

Location

URI:

Host:

Repository path:

Connection

Protocol:

Port:

Authentication

User:

Password:

☐ Store in Secure Store

 Import Projects from Git

Branch Selection

Select branches to clone from remote repository. Remote tracking branches will be created to track updates for these branches in the remote repository.



Branches of C:\Users\moise\OneDrive\Escritorio\Universidad\3er Año\IPS\PracticaGit\demo1:

☒  master

Tag fetching strategy

☒ When fetching a commit, also fetch its tags

☐ Fetch all tags and their commits

☐ Don't fetch any tags



Local Destination

Configure the local storage location for demo1.



Destination

Directory:

Initial branch: ▾

☐ Clone submodules

Configuration

Remote name:

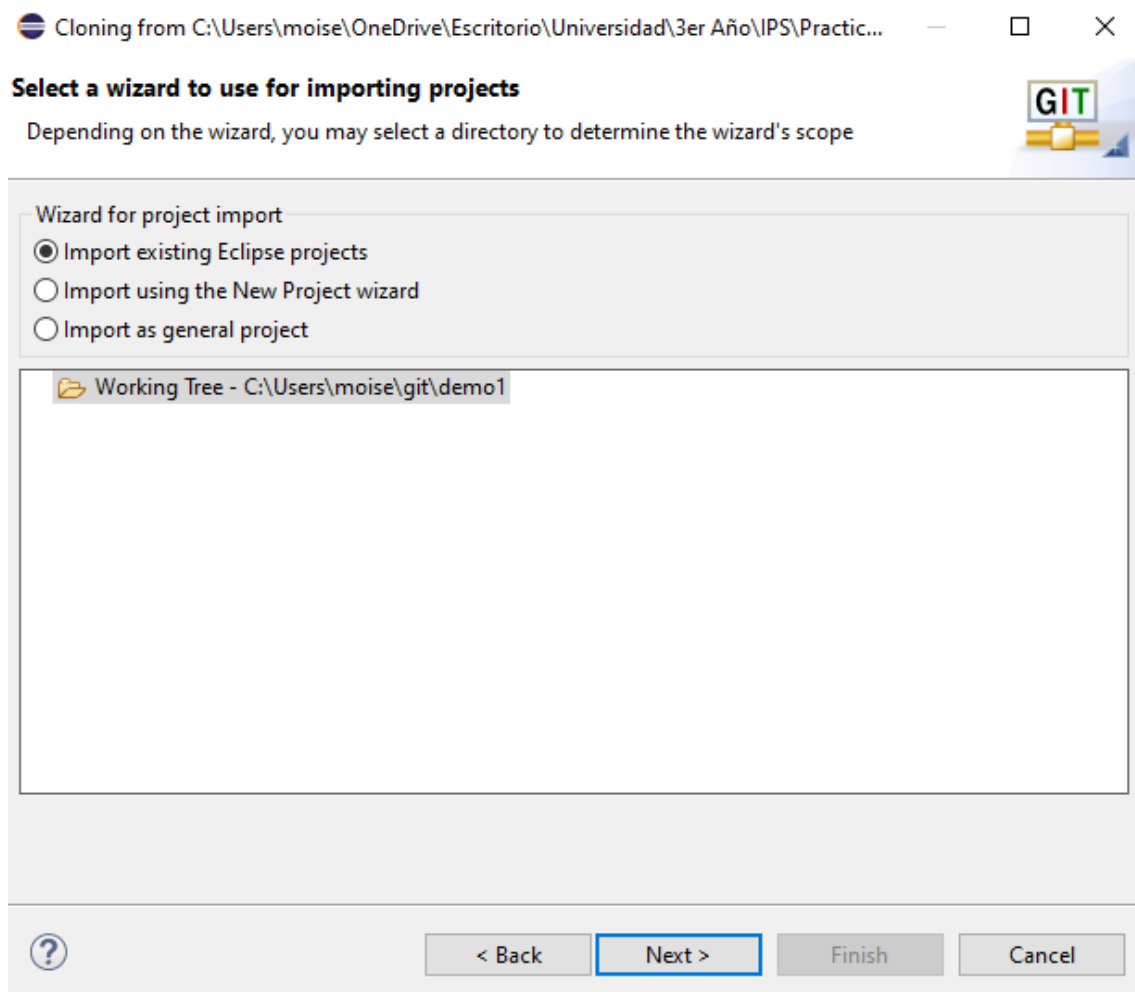


< Back

Next >

Finish

Cancel



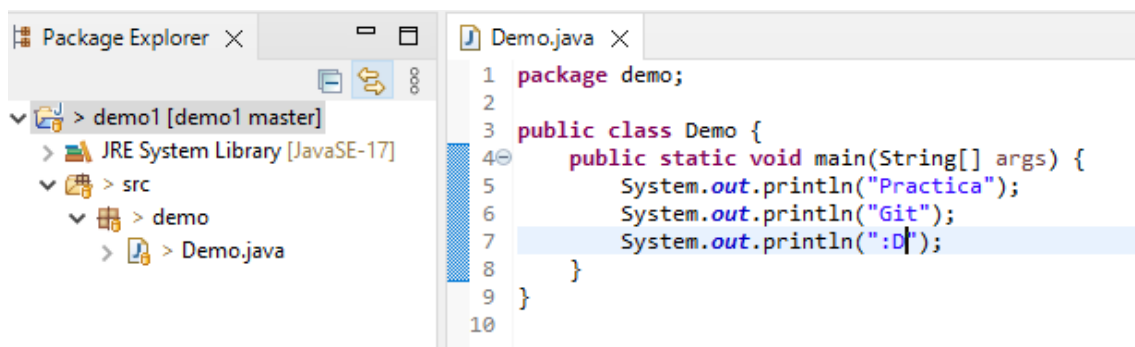
3. Realizando cambios desde un único usuario.

Commit: Envía cambios realizados en el workspace al repositorio local.

Push: Los envía al repositorio remoto.

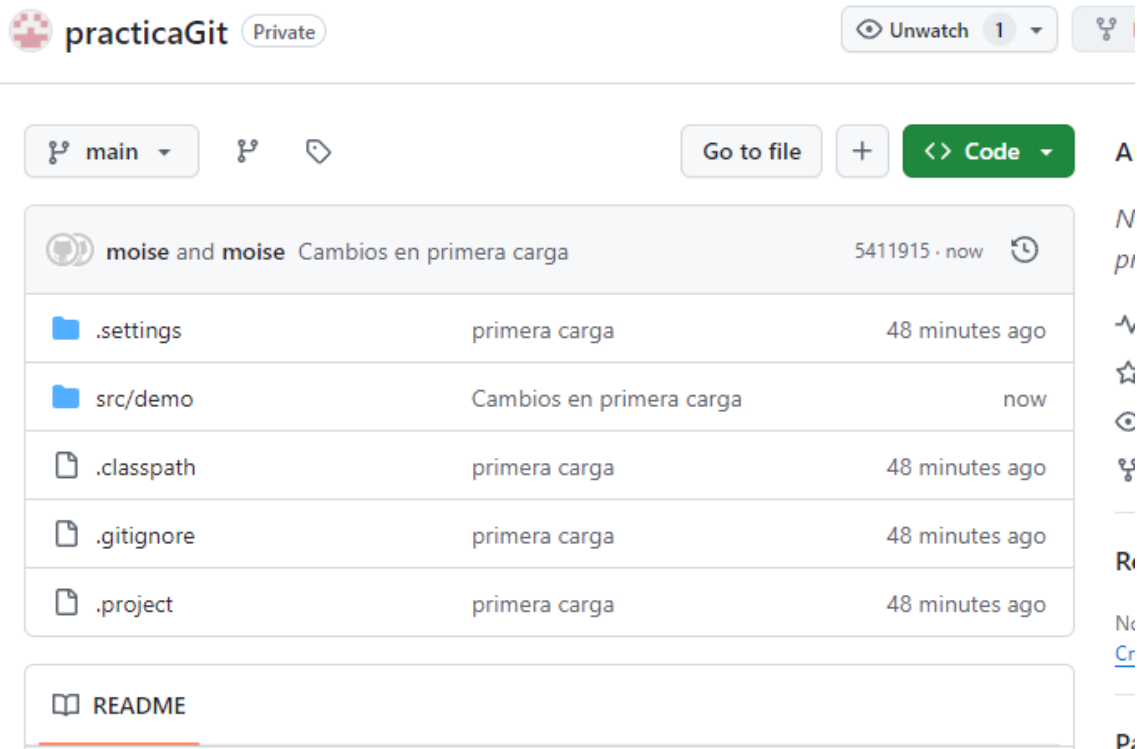
3.1. Commit y Push

Escribimos varias líneas de código sobre lo que queramos

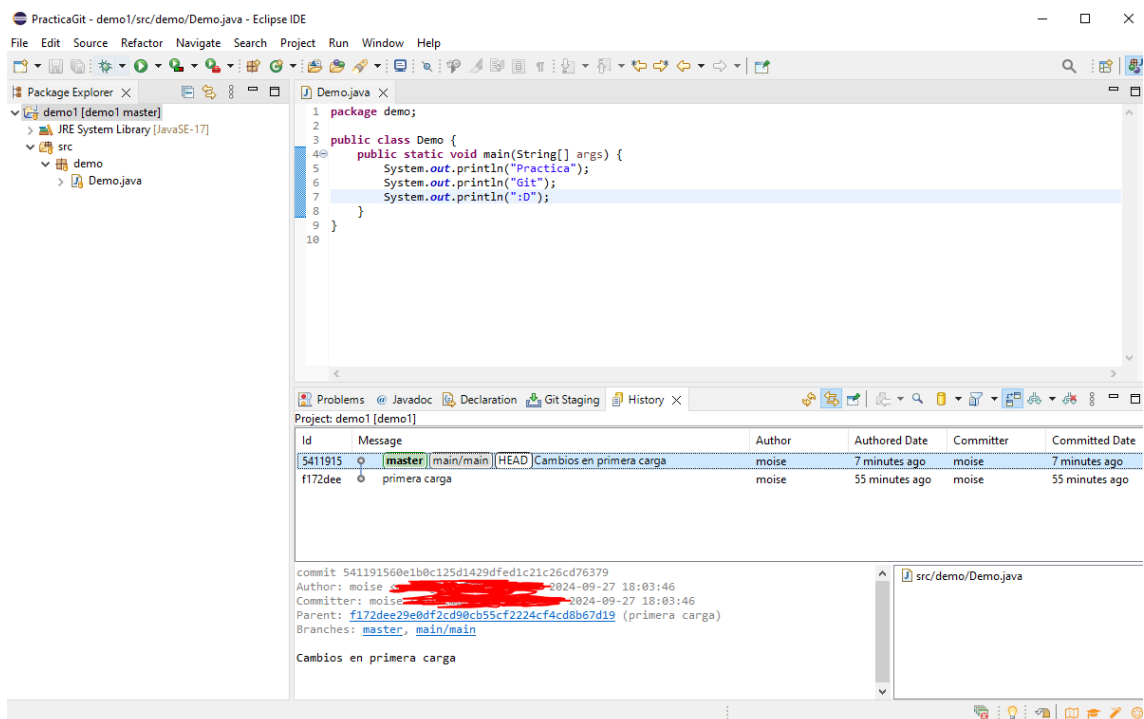


3.2. Examinando los cambios realizados

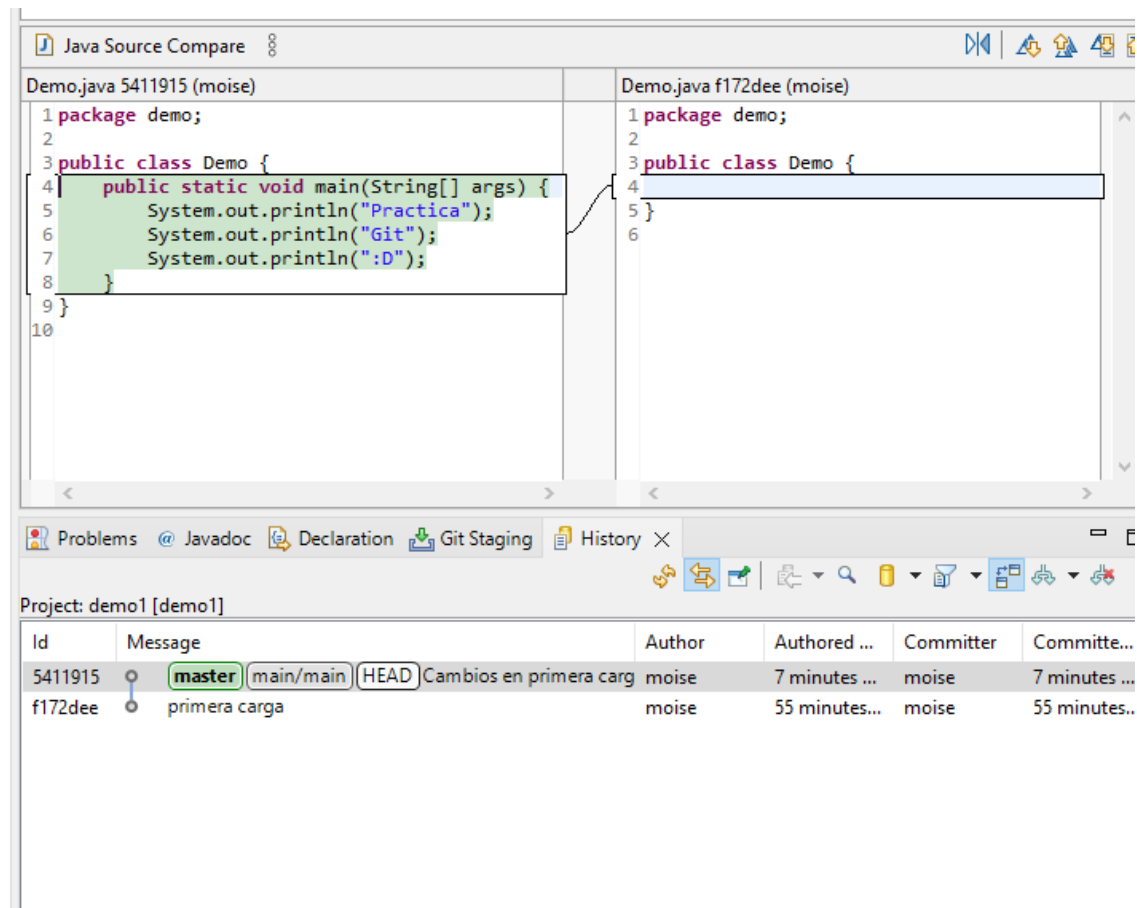
Subimos el archivo con los cambios que acabamos de realizar y observamos como aparece en el Git con el nombre “Cambios en primera carga”.



Comprobar los cambios también en la historia desde Eclipse.

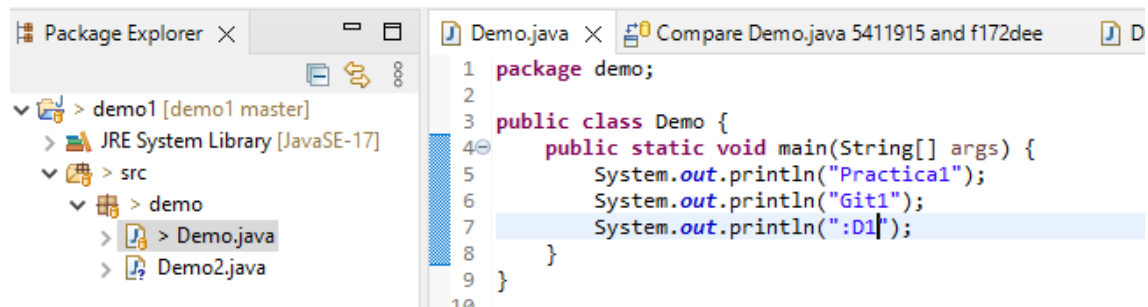


Visualizamos los cambios desde Eclipse, comparando las dos versiones.

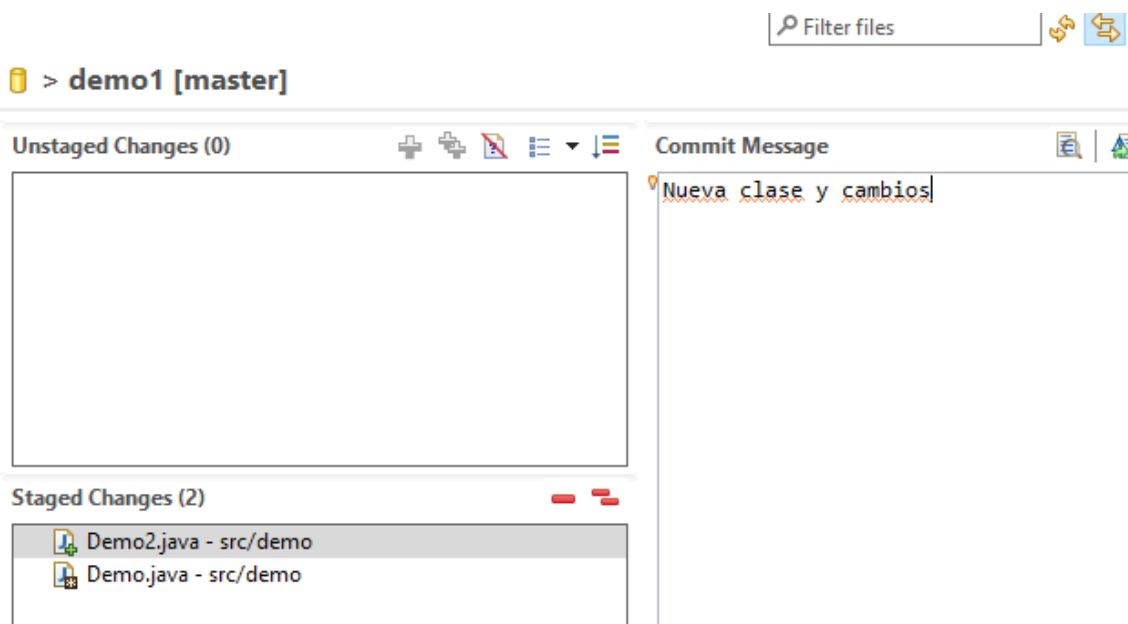


3.3. El índice de Git

Hacemos cambios en la clase Demo y creamos otra clase Demo2



Realizamos un Team→Commit actualizando en el repositorio los nuevos cambios

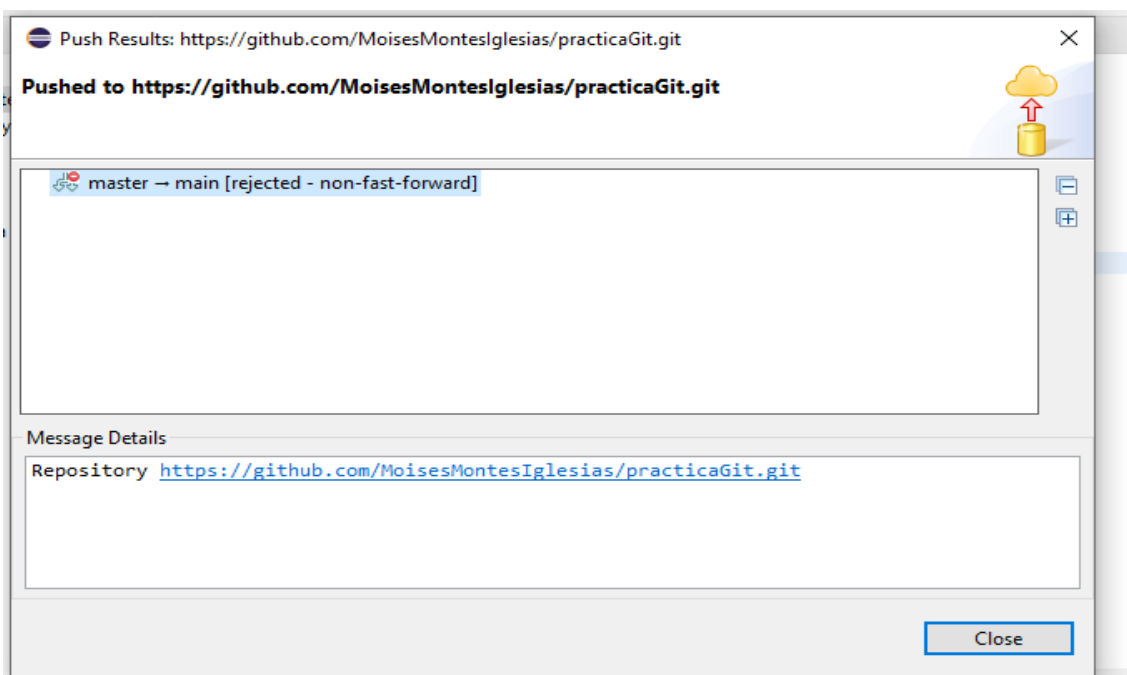


Los cambios que son enviados a un repositorio local con commit no son todos los que hemos hecho en el workspace, sino solamente los han sido previamente añadidos al índice de Git.

4. Cambios de varios usuarios en la misma rama

4.1. Commit y Push concurrentes

Cree otro workspace para poder hacer cambios en el mismo proyecto con “diferentes usuarios”. El commit se dice non-fast-forward porque trata de incorporar cambios que fueron realizados partiendo de una situación del repositorio anterior a la actual.

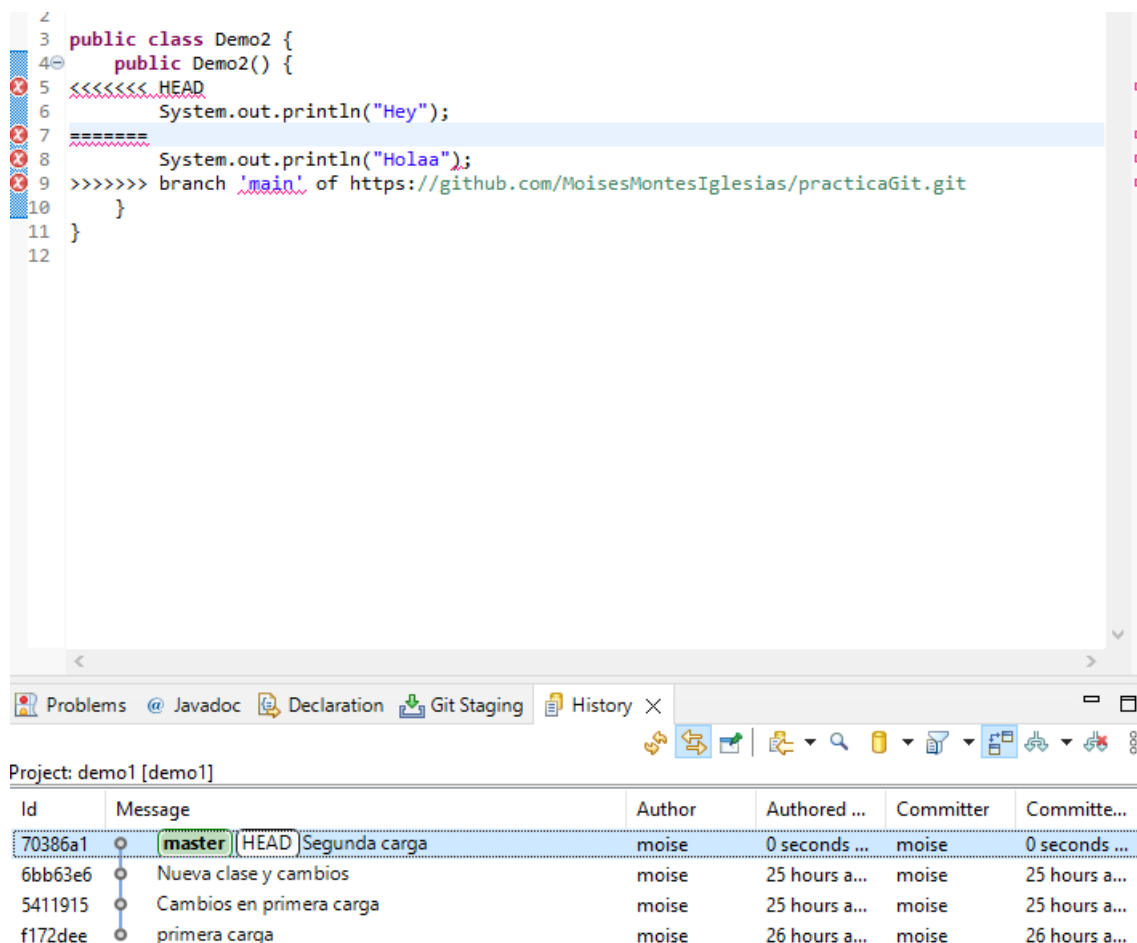


4.2. Actualización con los cambios de otro usuario (Pull)

Fetch: Trae desde un repositorio al workspace los últimos cambios realizados en el repositorio (ya sean realizados por nosotros o por otros usuarios).

Para mezclar nuestros cambios con los de los otros usuarios, empleamos el Pull, que realiza una combinación de Fetch y un Merge.

Merge: Operación que junta o mezcla los cambios realizados en dos ramas.



The screenshot shows an IDE window with a Java file named `Demo2.java`. The code contains a `public class Demo2` with a `public Demo2()` constructor. There is a merge conflict between the `HEAD` and the `main` branch of the repository `https://github.com/MoisesMontesIglesias/practicaGit.git`. The conflict is resolved in favor of the `main` branch, which prints "Holaa".

Below the code editor, the `History` tab is active, showing a list of commits for the project `demo1 [demo1]`.

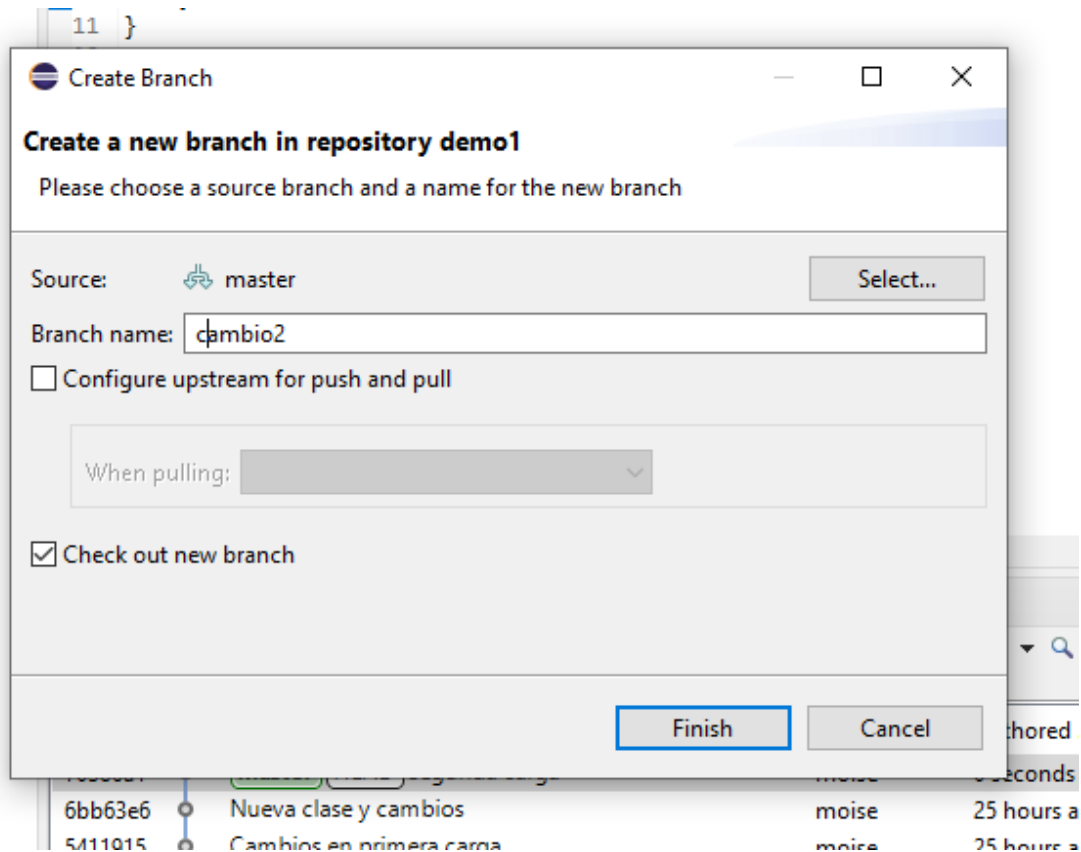
Id	Message	Author	Authored ...	Committer	Committe...
70386a1	[master] HEAD Segunda carga	moise	0 seconds ...	moise	0 seconds ...
6bb63e6	Nueva clase y cambios	moise	25 hours a...	moise	25 hours a...
5411915	Cambios en primera carga	moise	25 hours a...	moise	25 hours a...
f172dee	primera carga	moise	26 hours a...	moise	26 hours a...

5. Cambios de varios usuarios utilizando distintas ramas

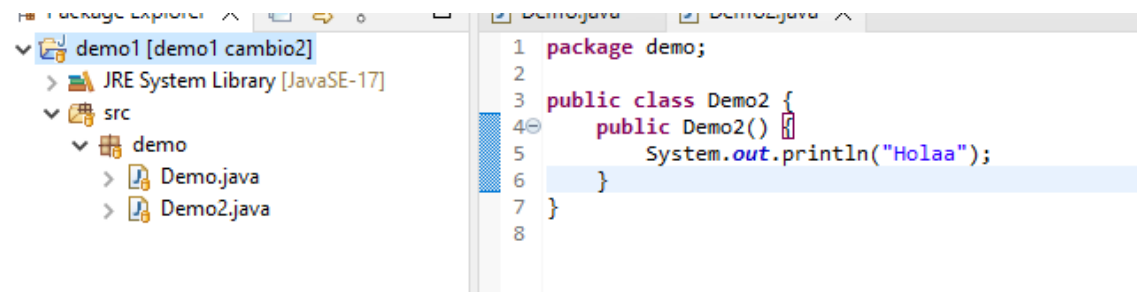
5.1. Trabajando en una rama independiente(checkout).

Creo una nueva rama `cambio2` que contiene lo mismo que nuestra rama `master` y nos situamos sobre ella.

IMPORTANTE: activar la opción "Check out new branch".



Observamos que estamos en la nueva rama que acabamos de crear.



2) Realizar cambios en la rama actual.

Estas ramas no son públicas, ya que no se encuentran en el repositorio remoto salvo que hagamos un Push de la rama, pero, todos los commit realizados se verán en el repositorio cuando hagamos un merge. Por lo tanto, hay que darle importancia al nombre o descripción de los cambios cuando hacemos los commit.

Project: demo1 [demo1]						
Id	Message	Author	Authored ...	Committer	Committe...	
5190bac	cambio2 main/cambio2 HEAD Más cambios	moise	10 second...	moise	10 second...	
421f411	Merge branch 'main' of https://github.com/Mois	moise	7 hours ago	moise	7 hours ago	
70386a1	master Segunda carga	moise	7 hours ago	moise	7 hours ago	
2f358f7	main/main segundo cambio	moise	7 hours ago	moise	7 hours ago	
bb2cd96	Usuario2	moise	32 hours a...	moise	32 hours a...	
6bb63e6	Nueva clase y cambios	moise	32 hours a...	moise	32 hours a...	
5411915	Cambios en primera carga	moise	32 hours a...	moise	32 hours a...	
f172dee	primera carga	moise	33 hours a...	moise	33 hours a...	

5.2. Juntando los cambios de diferentes ramas

Integramos los cambios realizados en la rama cambio2 en master.

Sin embargo, otro usuario nos ha realizado cambios en la rama master. Por lo tanto, debemos hacer lo siguiente:

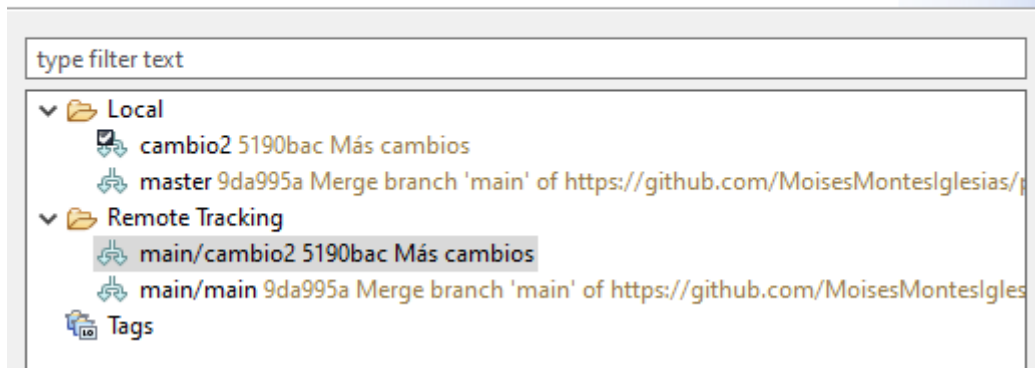
- 1) En master tenemos la última versión. Cambiamos a esta rama y actualizamos master con los cambios del otro usuario.
- 2) Debemos mantener la rama master lo más libre posible de versiones temporales o conflictivas.

Para ello, es más seguro integrar primero los cambios de otro usuario y tras solucionar los problemas o conflictos, integrar estos cambios en master.

Project: demo1 [demo1]						
Id	Message	Author	Authored ...	Committer	Committe...	
9da995a	master main/main HEAD Merge branch 'mai	moise	1 seconds ...	moise	1 seconds ...	
70386a1	Segunda carga	moise	7 hours ago	moise	7 hours ago	
2f358f7	segundo cambio	moise	7 hours ago	moise	7 hours ago	
bb2cd96	Usuario2	moise	32 hours a...	moise	32 hours a...	
6bb63e6	Nueva clase y cambios	moise	32 hours a...	moise	32 hours a...	
5411915	Cambios en primera carga	moise	32 hours a...	moise	32 hours a...	
f172dee	primera carga	moise	33 hours a...	moise	33 hours a...	

Merge 'cambio2'

Select a branch or tag to merge into the 'cambio2' branch



Project: demo1 [demo1]

Id	Message	Author	Authored ...	Committer	Committe...
8d1adfa	master main/main (HEAD) Merge branch 'x' moise	moise	2 seconds ...	moise	2 seconds ...
9da995a	Merge branch 'main' of https://github.com	moise	27 minute...	moise	27 minute...
5190bac	cambio2 main/cambio2 Más cambios	moise	40 minute...	moise	40 minute...
421f411	Merge branch 'main' of https://github.com	moise	8 hours ago	moise	8 hours ago
70386a1	Segunda carga	moise	8 hours ago	moise	8 hours ago
2f358f7	segundo cambio	moise	8 hours ago	moise	8 hours ago
bb2cd96	Usuario2	moise	32 hours a...	moise	32 hours a...
6bb63e6	Nueva clase y cambios	moise	32 hours a...	moise	32 hours a...
5411915	Cambios en primera carga	moise	33 hours a...	moise	33 hours a...

commit 8d1adfa945831e0dc94a24ece42a293a4b25332e

3) Que el merge se realice sin problemas no implica que esté bien, ya que puede no funcionar y/o compilar. Debemos probar los cambios.

4) Debemos realizar un Push Branch 'master' para enviar los cambios del repositorio local al remoto.

5.3. Solucion de conflictos en merge

Creamos una rama cambio3 en la que modificamos algunas líneas.

Mientras, otro usuario edito la rama master del repositorio remoto, el cual edito la misma línea que nosotros.

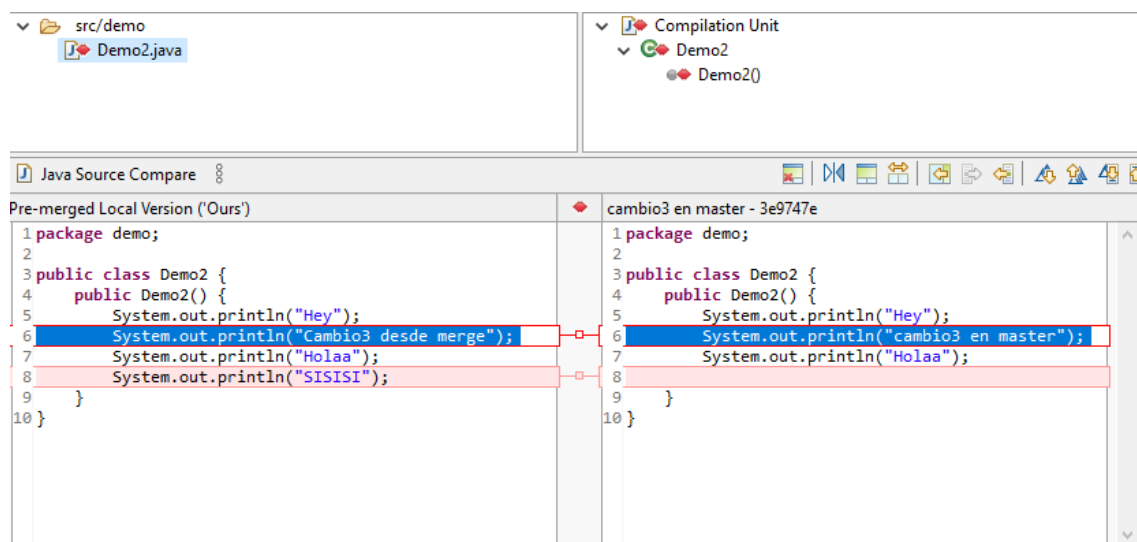
En este caso, tras hacer el merge de la rama cambio3 con master. Nos aparece un conflicto como el generado en la imagen, en el que vemos las dos versiones en conflicto.

```

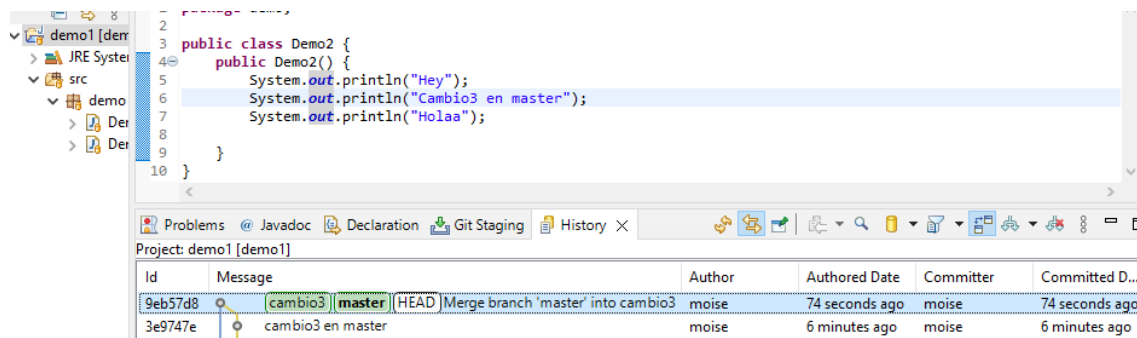
4
3 public class Demo2 {
4     public Demo2() {
5         System.out.println("Hey");
6         <<<<<<< HEAD
7             System.out.println("Cambio3 desde merge");
8         =====
9             System.out.println("cambio3 en master");
10        >>>>>> refs/heads/master
11        System.out.println("Holaa");
12        <<<<<<< HEAD
13        System.out.println("SISISI");
14        =====
15
16        >>>>>> refs/heads/master
17    }
18 }

```

Si empleamos para visualizar los conflictos Merge Tool, se vería de esta manera:



Realizamos un nuevo commit tras solucionar los conflictos al Git



6. Manteniendo limpio y ordenado el historial de cambios

Realizando diversos commits para cambios parciales sobre la implementación de la funcionalidad o para solucionar conflictos surgidos. Realizar todos estos commits, supone ensuciar la rama, ya que tendría muchos commits irrelevantes. Lo ideal sería registrar

un único commit en master que refleja todo el trabajo realizado para implementar la historia de usuario.

Para ello usaremos dos técnicas:

- 1) Rebase: Cuyo objetivo es reordenar los commits
- 2) Squash: Su objetivo es juntar varios commits como si fueran uno solo

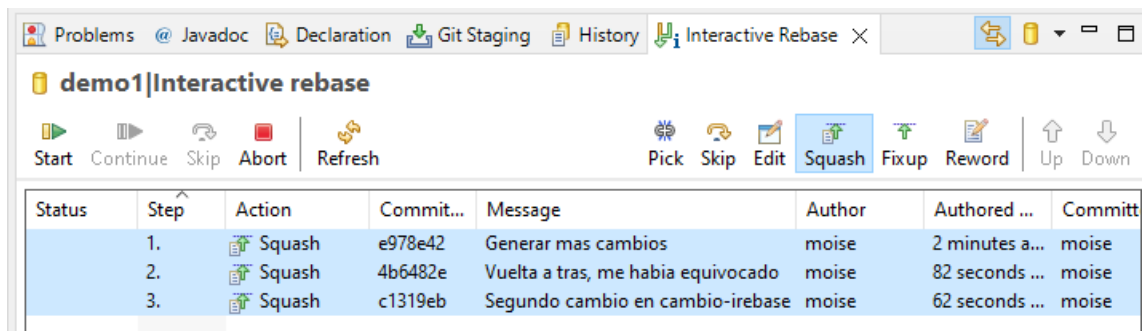
6.1. Agrupando commits: Interactive Rebase

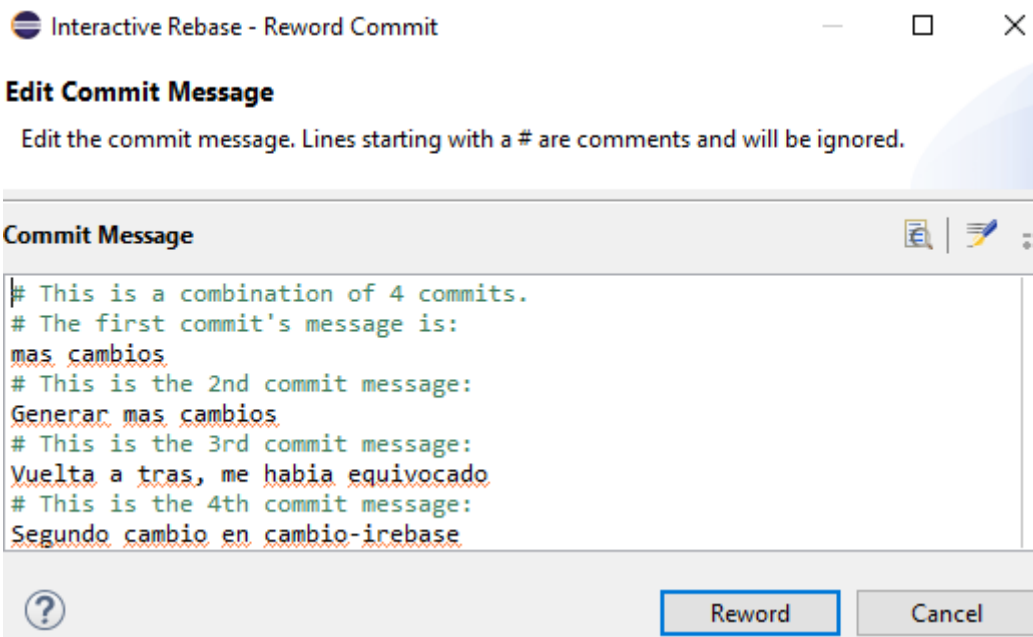
Rebase permite tener una historia más ordenada pero no evita que queden sucias con commits de poco interés.

Supongamos que en nuestra rama hemos realizado 4 cambios que no queremos que se reflejen en master, para ello empleamos rebase interactivo.

El rebase interactivo permite alterar la historia de nuestra rama, usando squash que permite juntar varios commits en uno solo.

Seleccionamos los commits que queremos modificar. Tras seleccionarlos todos los que nos interesan, hacemos squash y nos saldrá una ventana donde se muestra un mensaje que reemplazará a los que hemos seleccionado.





Tras revisar el historial vemos como el último commit ha sustituido los anteriores. Ahora en master se refleja un único commit en vez de 4.

Id	Message	Author	Authored ...	Committer	Committe...
a7a6780	# This is a combination of 4 commi	moise	7 minutes ...	moise	1 seconds ...
f1267f0	Mas y mas cambios	moise	8 minutes ...	moise	8 minutes ...

6.2. Merge squash

También podemos hacer un merge en master de forma que todos los cambios realizados en nuestra rama se vean como uno solo.

La operación merge squash permite incorporar en el workspace los cambios realizados en otra rama. Vamos directamente a master y hacemos un merge squash con la rama cambio2-mergesquash.

Tras este merge, tendremos en el workspace los cambios realizados en la rama cambia2-mergesquash (debemos hacer un commit posteriormente).



OJO: Al realizar este proceso, perdemos la relación entre las ramas que estamos juntando. Ya que realiza un proceso similar al hacer los cambios directamente sobre la rama master.